Portfolio.

로봇을 움직이는 재미에 빠진
로봇 SW 개발자

황인재

EMAIL
il1282113@gmail.com

PHONE
010-5377-1314

GITHUB
github.com/hwang-injae

LINKEDIN
linkedin.com/in/hwang-injae

Contents.

01 **About me.** 나에 대해 — 03
배경 · 일하는 방식

02 **Profile.** 프로필 — 04
핵심역량 · 교육 · 프로젝트 · 기술

03 **Project 1.** BuildMind — AI 설계 블록 조립 협동로봇 — 05 – 07
팀장(PL) · 웹 화면 · AI 설계 생성 · DB 설계 · YOLO 학습

04 **Project 2.** SortTwin — AMR과 협동로봇이 택배를 분류하는 디지털 트윈 — 08 – 10
P3020 Pick & Place · 비전 연동

05 **Project 3.** PreWash-Cell — 다회용기 예비세척 자동화 셀 — 11 – 13
프로젝트 관리(PM) · 인터페이스 설계 · 운영 화면 · 기록 DB · 셀 통합 · 무게 측정

06 **Others.** 그 밖의 프로젝트 — 14
IDC Patrol · 학습 프로젝트 · 실습

07 **Thank you.** 마무리 — 15
맺음말 · 연락처



About me.

여러 길을 돌아
로봇 SW에 자리를 잡았습니다

편입 준비, 전과, 창업동아리까지 여러 길을 거치며 제게 맞는 일을 찾아다녔습니다.

그러다 스타트업에서 AI 서비스를 써 보며 이를 로봇에 접목하고 싶어졌고,

부트캠프에 들어갔습니다. 머릿속에 있던 기능이 실제 로봇에서

처음 움직이던 순간, 로봇을 움직이는 재미에 빠졌습니다.

그 뒤로 매일 아침 일찍 교육장에 나가 로봇을 만졌습니다.

많이 아는 것보다 새로운 기술을 우리 상황에 맞게 빨리 써 보는 쪽에

무게를 두고 일했습니다. 배운 것을 바로 써 보고,

막히면 원인이 드러날 때까지 하나씩 확인했습니다.

일하는 방식

적용	**새로 배운 기술을 우리 상황에 어떻게 쓸지부터 생각합니다** 다른 팀 발표에서 본 three.js를 BuildMind 설계도 3D 미리보기에 적용 · IDC Patrol에서 배운 "인터페이스 먼저 고정"을 PreWash-Cell 함수 계약에 적용
병렬	**로봇 없이도 rosbag과 모의 드라이버로 병렬 개발합니다** Isaac Sim 색상 분류 실습에서 녹화한 카메라 rosbag으로 판별을 먼저 확인한 뒤 시뮬레이터에 연동 · PreWash-Cell에서 모의(Mock) 드라이버로 4명이 로봇 없이 개발
원인	**문제가 생기면 코드 밖까지 의심 지점을 하나씩 지워 나갑니다** PreWash-Cell에서 측정 조건을 바꿔 가며 무게 오차의 원인을 케이블 장력으로 좁혀 90 g → +3 g · 콜백 안에서 로봇 명령이 멈추는 문제는 모의(Mock) 드라이버로 재현해 실행 구조가 원인임을 확정
검증	**결과는 기록과 실제 동작으로 확인한 뒤에 반영합니다** BuildMind 팀장(PL)으로 결정 사항 123건을 기록하고, CI를 통과한 PR만 순서대로 병합한 뒤 통합 시험과 리허설로 동작을 확인해 설계 생성부터 로봇 조립까지 시연

Profile.

황인재 — 로봇 SW 개발 · 신입

협동로봇(M0609 · P3020)과 모바일 로봇(TurtleBot4)으로 센서 · 비전 판단 결과를 로봇 동작과 운영 화면으로 잇는 프로젝트 4건을 했습니다.

핵심역량

시스템 통합	Pick & Place 상태 흐름 · Action 서버, 로봇–웹 PC 토픽 규약(MQTT), 요청부터 동작까지 전체 흐름 통합
실기 디버깅	두산 API · 두산 로봇 언어(DRL) → ROS 2 변환, 값이 이상하면 측정 조건과 실행 환경부터 확인해 원인을 좁힘
비전 · AI 연동	윤곽 분할(YOLO seg) 학습과 라벨링 기준, 검출 픽셀과 깊이로 3D 좌표 변환, 대규모 언어 모델(LLM) 구조화 출력
디지털 트윈	Isaac Sim에서 Pick & Place 구성, 카메라 좌표와 로봇 좌표 차이 같은 통합 문제를 실기 전에 발견
운영 화면 · 기록	FastAPI · Next.js 화면, PostgreSQL · SQLite 기록, 모의(Mock) 구현으로 로봇 없이 자동 시험

교육 · 학력

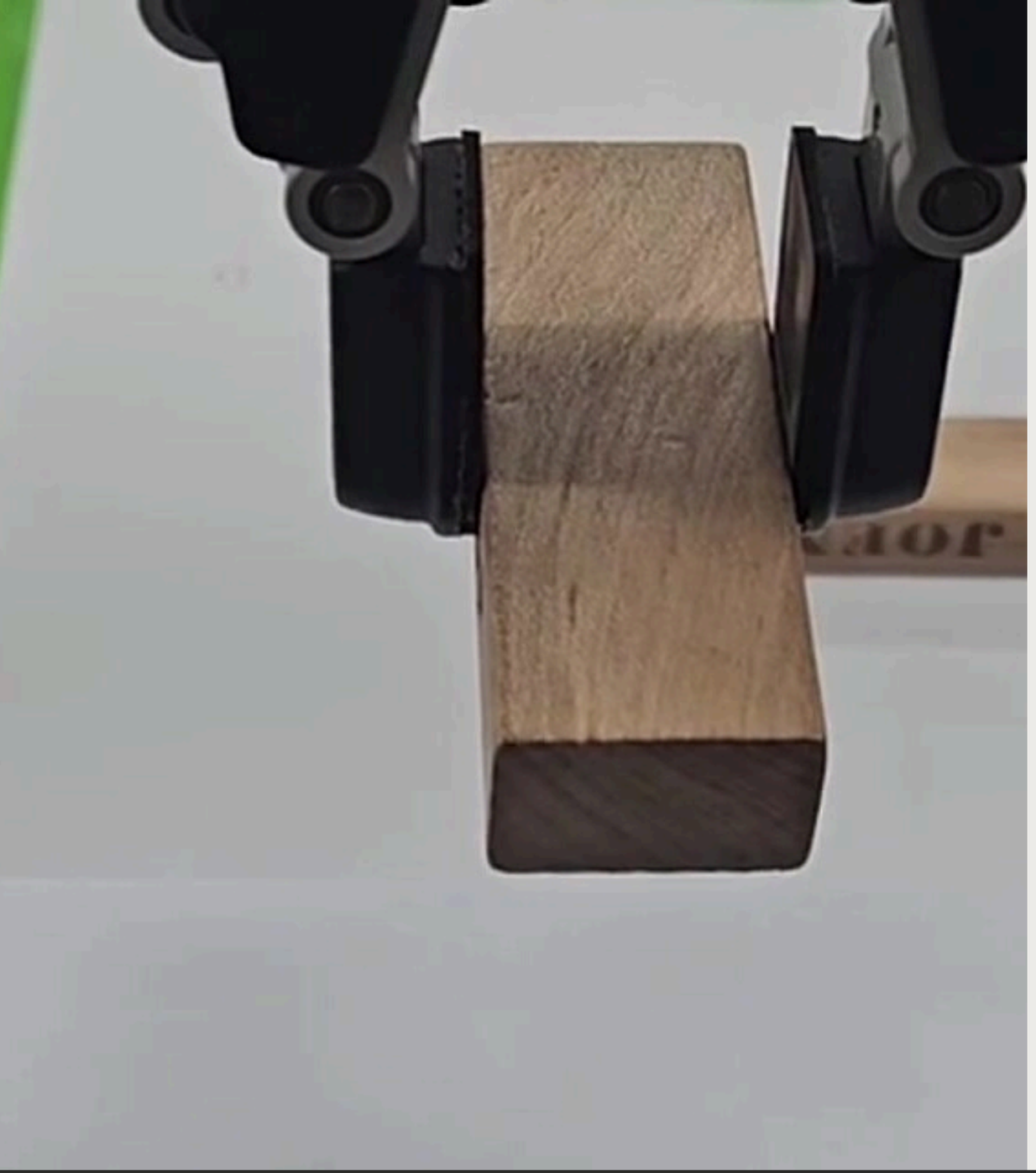
2026.04 – 2026.10	두산로보틱스 ROKEY 부트캠프 9기 — AI 로보틱스 엔지니어 양성과정 수료 이론 4개월 · 프로젝트 2개월
2017.03 – 2027.08	국립부경대학교 화학과 졸업 예정

프로젝트

BuildMind	AI 설계 블록 조립 협동로봇 · 2026.10 · 5인 · PL 블록 집기 98% · AI 설계 형식 실패 0건
SortTwin	AMR × 협동로봇 택배 분류 디지털 트윈 · 2026.08 · 4인 · Pick & Place · 비전 연동 입고 → 분류 1미션 · Pick & Place 8단계
PreWash-Cell	다회용기 예비세척 자동화 셀 · 2026.09 · 4인 · PM 시작 1번으로 무정지 완주 약 13분 · 수락 기준 7/7
Others	IDC Patrol — TurtleBot4 2대 데이터센터 순찰 · 2026.09 · 8인 · 데이터 기준 → PM 손 제스처 추적 로봇 · M0609 기어 DRL → ROS 2 · TurtleBot4 RC카 탐색 · 추적

기술

Python · ROS 2 Jazzy · 두산 M0609 · P3020 · TurtleBot4 · Nav2 · Isaac Sim 5.1 · Lula IK · YOLO seg · OpenCV · RealSense · RGB-D · FastAPI · Next.js · PostgreSQL · SQLite · MQTT · Docker · Git · pytest



Project 1.

말로 설계하고, 로봇이 조립한다
BuildMind

가구 모양이 바뀔 때마다 사람이 설계와 로봇 교시를 다시 해야 하는 일을 줄이기 위해, 블록 가구(벤치 · 의자 · 책상) 조립으로 그 공정을 모사한 시스템입니다. 사용자가 말이나 글로 요청하면 AI가 블록 설계도를 만들고, 협동로봇이 흩어진 블록을 찾아 조립합니다.

11/11
설계대로 조립한 블록

98%
흩어진 블록 집기 성공률

0.972
블록 인식 mAP50 · 0.87에서 개선

0건
AI 설계 형식 실패

기간 · 팀	2026.10.01 – 10.16 · 5인
담당	팀장(PL) · 웹 화면 · AI 설계 생성 · DB 설계 · YOLO 학습
기술	ROS 2 · MoveIt2 · 두산 M0609 + RG2 · RealSense D435i · YOLO seg · Claude · FastAPI · Next.js · PostgreSQL · MQTT · Docker
링크	github.com/hwang-injae/buildmind · README에 1분 시연 영상

What I did.

AI 설계 · DB · 웹 화면부터
YOLO 학습과 팀 통합까지 맡았습니다

AI 설계

AI 응답을 JSON Schema 형식으로만 받아 설계 후보 3개 생성 → 형식 실패 0건

조립 가능 검사(무게중심 · 받침 · 작업 영역 등 8단계)를 통과하지 못하면 실패 원인을 다시 전달해 재생성 · 최대 2회 · 전체 5분 제한

DB 설계

설계를 부모–버전으로 이어 저장하는 세대 관리 → 수정 요청에 따라 파생 설계 15개를 3세대까지 추적

PostgreSQL 테이블 3개(설계 · 조립 · AI 호출 기록) · 같은 템플릿의 이전 설계를 DB에서 최대 2개 조회해 AI에 전달하는 검색 증강 생성(RAG)

웹 화면

설계 3D 미리보기 · 로봇 카메라 · 조립/정지 버튼을 한 화면에

Next.js · three.js · 로봇 PC와는 MQTT 한 경로로만 통신하도록 규약 설계

YOLO

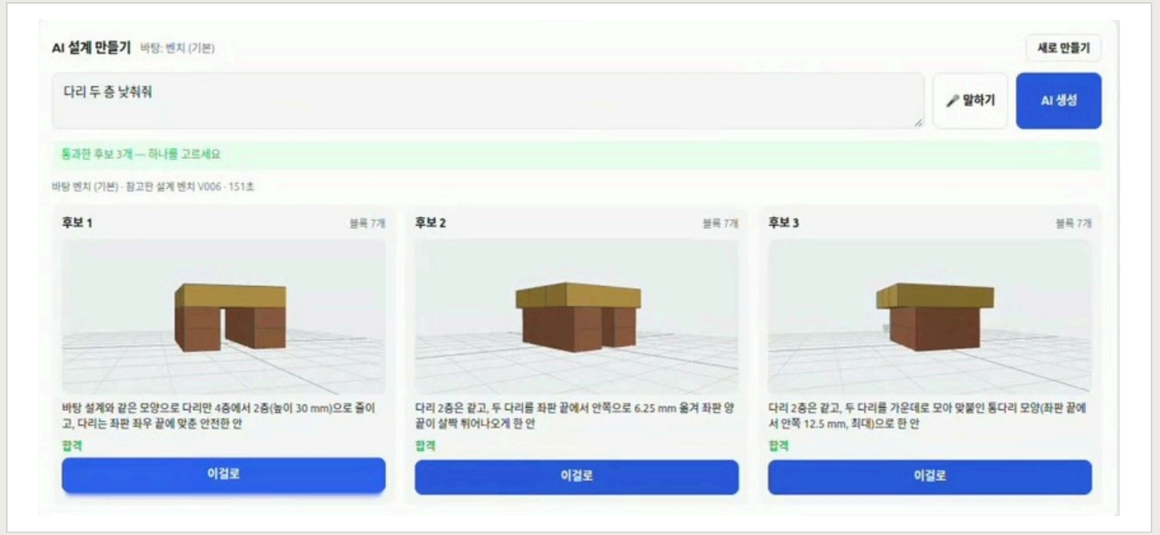
블록 인식 모델 학습 — 사진 208장 라벨링 · 모델 5종 비교 → 검출 성능 (mAP50) 0.87 → 0.972

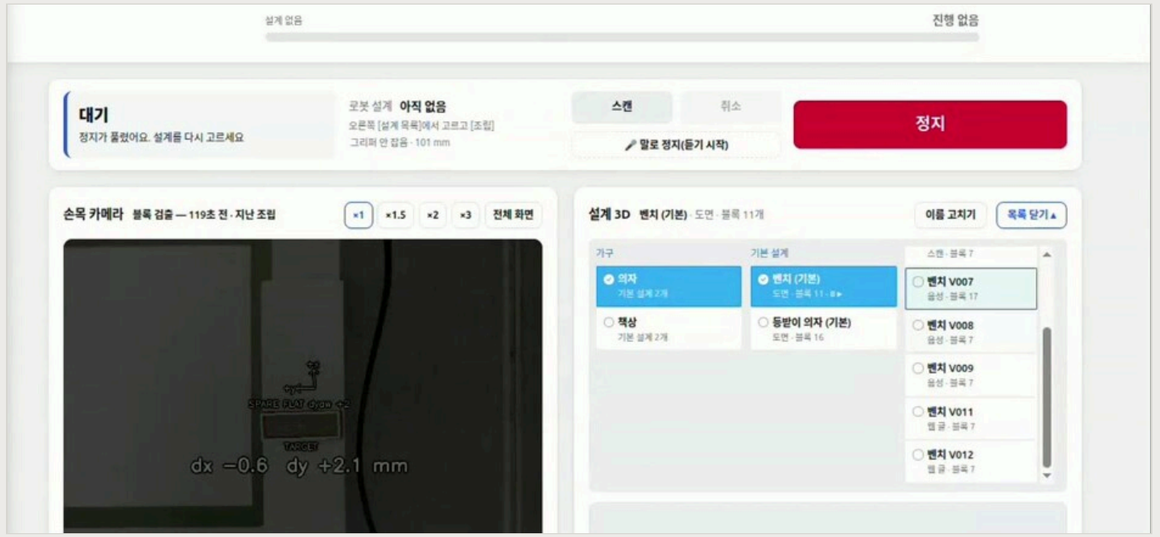
흩어진 블록을 찾는 데 사용

팀장

코드 병합 요청(PR)을 관리하고 결정 사항 123건을 기록, 기능 동결 뒤에는 CI를 통과한 PR만 통합 브랜치에 순서대로 병합 → 통합 시험 · 리허설 완주

요구사항 · 인터페이스 · 설계 문서, 일정, 발표





위 — AI가 만든 설계 후보 3개와 검사 결과.
아래 — 웹 화면: 로봇 카메라 · 설계 3D 미리보기 · 정지 버튼.

Problem.

쌓인 블록을
YOLO가 놓치는 문제

문제

블록이 겹친 장면에서 인식률이 떨어짐 — 검출 성능(mAP50) 0.87, 검증 48장에서 놓친 블록 100개 이상

흩어진 블록은 잘 찾았지만 가려진 블록은 자주 놓침

원인

가려진 블록이 두 조각으로 보일 때 라벨 기준이 사람마다 달랐음

같은 경우를 한 개 또는 두 개로 섞어 표시해 모델이 일관되게 배우지 못함 · 후처리로는 해결 안 됨

해결

라벨 규칙을 하나로 통일해 다시 라벨링 · 재학습하고, 모델 5종을 비교해 최종 모델 채택

큰 조각 하나만 블록으로 표시 · 작은 조각은 제외 · 검증 데이터에도 같은 규칙

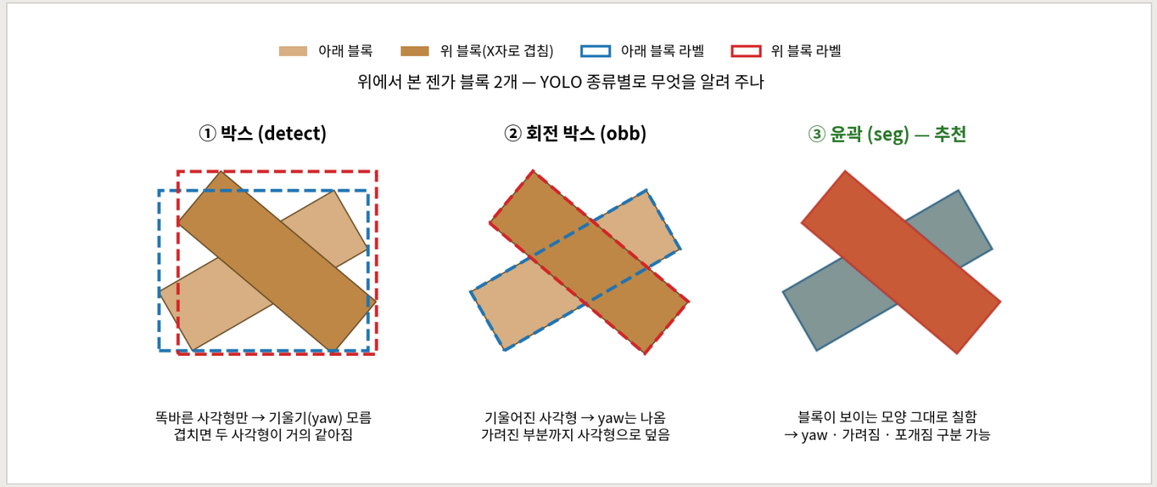
결과

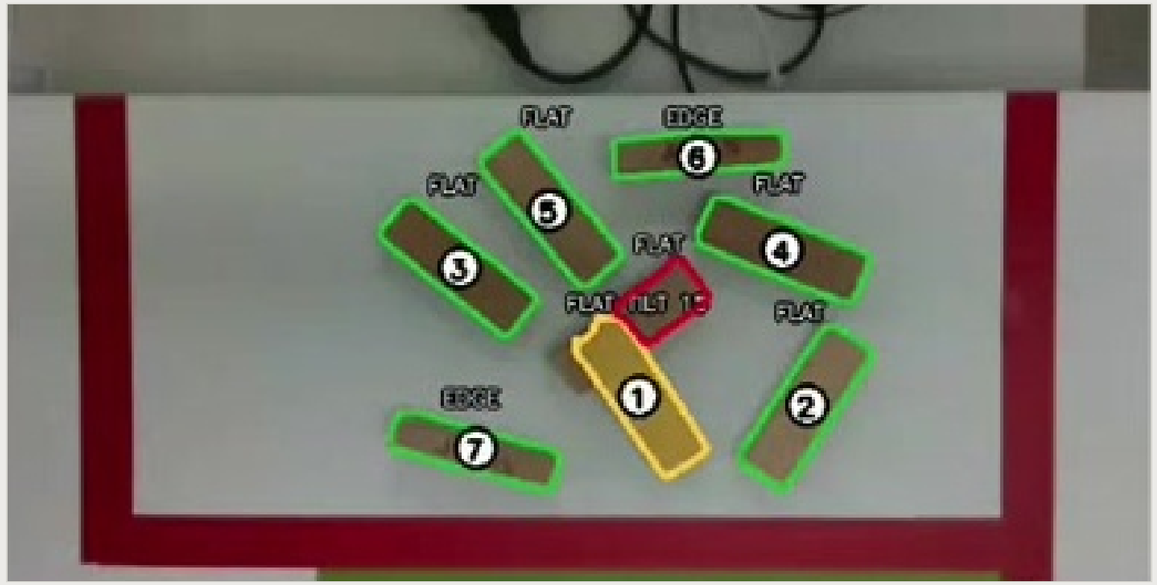
검출 성능(mAP50) 0.87 → 0.972 · 통합 시험 집기 성공률 98%

비교한 5종 중 채택한 yolo26s-seg · 150 epoch · 사진 208장

배운 점

- 앞선 IDC Patrol에서 배운 "모델을 바꾸기 전에 **데이터 기준부터 통일**"을 그대로 적용했고, 라벨 규칙 하나를 맞추자 가려진 블록까지 잡혔습니다. 사람마다 다른 라벨 기준이 모델 성능을 떨어뜨린다는 것도 이때 확인했습니다





위 — 모델 종류 비교: 겹친 블록은 윤곽(seg) 방식이어야 기울기와 가려짐을 구분할 수 있음.
아래 — 겹친 블록을 로봇 카메라가 인식한 화면: 초록 = 집을 수 있음 · 노랑 = 다른 블록 위 · 빨강 = 덮여 있음 (색 판정은 비전 파트의 후처리).



Project 2.

AMR과 협동로봇이 택배를 나르고 분류하는 디지털 트윈 — SortTwin

입고 AMR(자율주행 로봇)이 박스를 싣고 오면 협동로봇(P3020)이 카메라로 박스를 찾아 컨베이어에 올리고, 휠 소터가 지역별로 분류한 뒤 불량품은 출고 로봇이 적재하는 물류 공정을 Isaac Sim 시뮬레이션에 구현했습니다(불량품 적재는 제출 뒤 팀에서 보완). 실제 장비 통합 전에 문제를 미리 찾는 것이 목적이었습니다.

1 미션
입고 → 상차 → 분류 한 미션으로 연결

8 단계
Pick & Place 상태 흐름

2 개
프로세스로 분리 · 표준 타입 토픽 2개로만 연결

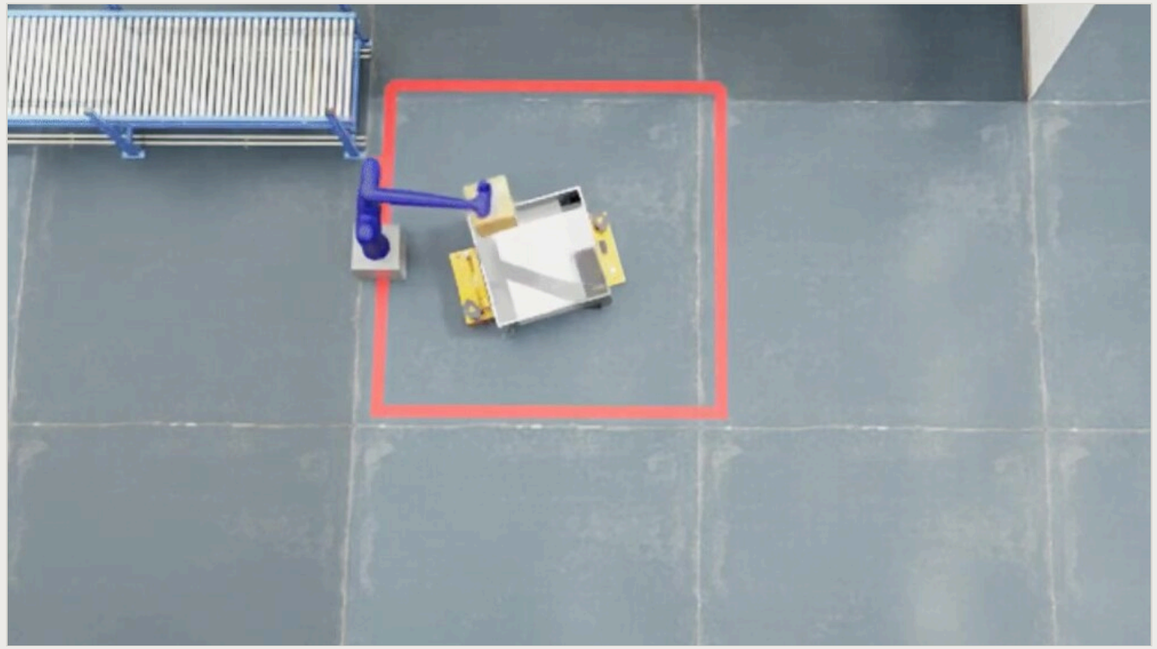
3 단
오검출 필터 · 거리 · 색 · 높이

기간 · 팀	2026.08.14 – 08.28 · 4인
담당	P3020 Pick & Place(입고 · 출고 첫 버전) · 카메라 인식 결과와 로봇 동작 연동
기술	NVIDIA Isaac Sim 5.1 · ROS 2 Jazzy · Lula IK · ROS 2 Action · RGB-D 카메라 · Python
링크	github.com/hwang-injae/amr-cobot-sorting · README에 전체 파이프라인 영상

What I did.

카메라가 본 박스를
로봇팔이 집어 옮기게 했습니다

입고 로봇	**접근 → 하강 → 잡기 → 들기 → 이동 → 내리기 → 놓기 → 완료의 8단계 상태 흐름으로 Pick & Place 구현 → 박스 바닥 기준 이송 높이로 벽 · 컨베이어 충돌 없이 이송** 역기구학(Lula IK) + 툴 끝 오프셋 보정 · 관절 안전 범위 제한 · 박스 방향(yaw) 자동 계산
비전 연동	**카메라 픽셀 + 깊이를 3D 좌표로 바꿔 박스 위치 계산, 오검출은 거리 · 색 · 높이 3단 필터로 제거 → 노란 AMR 몸체 오검출 제거** 중간 높이에서 한 번 더 검출해 먼 거리 픽셀 오차를 줄임
반복 처리	**박스 수를 미리 정하지 않고 화물이 빌 때까지 반복 → 5초 미검출이면 빈 화물로 판정해 AMR 복귀 요청** 박스 단위 진행 상황은 피드백으로, 종료는 신호 하나로 분리
출고 로봇	**불량품을 2 × 2 칸에 먼 칸부터 채우는 출고 노드 첫 버전 → 어느 칸에 놓을지 정하는 팀원의 코드를 로봇 동작에 처음 연결**
액션 서버	**수 분 걸리는 작업이라 ROS 2 Action으로 만들고 단계별 진행률을 피드백으로 전달 → goal 수락 · 진행률 피드백 정상** 흡착 그리퍼(VGP20) 동작 모사 포함



위에서 본 입고 구역 — AMR 위 박스를 협동로봇 P3020이 집어 컨베이어로 옮기는 중.

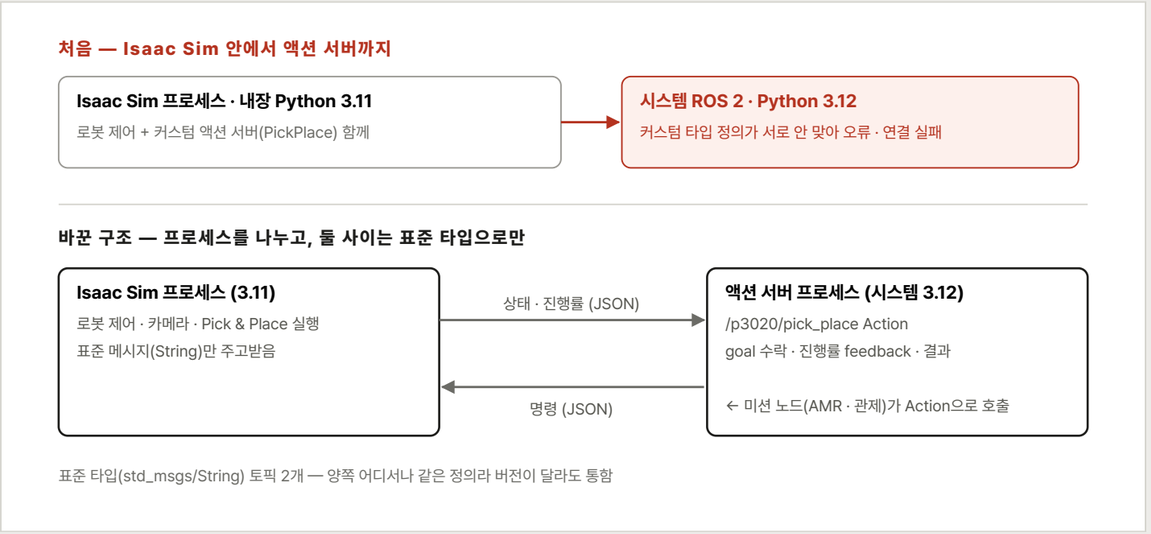
Problem.

시뮬레이터 안에서 만든 액션 서버가 ROS 2와 연결되지 않는 문제

문제	**Isaac Sim 안에서 Pick & Place 액션 서버를 만들자 타입 지원 오류로 연결 실패** "Type support not from this implementation" · goal을 보내도 받지 못해 액션 설계 자체가 막힘
원인	**Isaac Sim 내장 Python(3.11)과 시스템 ROS 2(Python 3.12)의 메시지 타입 지원이 서로 다름** 커스텀 액션 타입은 한쪽 버전으로만 빌드되어 다른 쪽에서 읽을 수 없음 · 표준 타입은 양쪽에 같은 정의가 있음
검토	**Docker는 Isaac Sim 쪽 인터프리터가 그대로라 해결이 안 되고, 내장 ROS 2 라이브러리 재빌드는 기한 안에 검증까지 마치기 어렵다고 판단해 보류**
해결	**액션 서버를 시스템 Python 프로세스로 분리하고, Isaac Sim과는 표준 타입 토픽 2개로만 연결** 명령과 상태를 JSON 문자열로 주고받아 커스텀 타입이 경계를 넘지 않게 함
결과	**goal 수락 · 단계별 진행률 피드백 정상 → 입고부터 분류까지 한 미션으로 연결**

배운 점

- 시뮬레이터에 내장된 Python과 PC의 **ROS 2 버전을 먼저 확인**해야 합니다. 커스텀 타입은 두 환경의 경계를 넘기지 않는 쪽이 안전합니다
- 카메라 좌표와 로봇 좌표의 차이처럼 **부분을 합칠 때만 드러나는 문제**는 디지털 트윈에서 실기 전에 찾을 수 있었습니다



처음 구조와 바꾼 구조 — 프로세스를 나누고 표준 타입으로만 연결.



Project 3.

카메라 없이, 협동로봇이 다회용기를 닦아 세척기에 꽂는다 — PreWash-Cell

반납된 다회용기를 협동로봇 1대가 집기 → 잔반 판정 → 닦기 → 헹굼 → 팔레트 적재까지 시작 버튼 한 번으로 처리하는 자동화 셀입니다. 카메라 없이 그리퍼 파지 폭 · 무게 · 힘 세 신호로만 판단하고, 용기마다 처리 기록을 남깁니다.

약 13분
시작 1번으로 무정지 완주
그릇 2 · 컵 2 · 사람 개입 0

7/7
미리 정한 수락 기준
7개 모두 충족

7종
예외 상황 실기 복구
툴 놓침 · 케이블 이상 · 정지/재개 등

4/4
잔반 용기 4회 중 4회 감지
빈 용기 8회 중 오판 0

기간 · 팀	2026.09.18 – 09.30 · 4인
담당	프로젝트 관리(PM) · 인터페이스 설계 · 운영 화면 · 기록 DB · 셀 통합 · 무게 측정
기술	ROS 2 · 두산 M0609 + RG2 · 힘 제어 · FastAPI · Next.js · SQLite · Fast DDS
링크	github.com/hwang-injae/prewash-cell · README에 2분 시연 영상

What I did.

인터페이스를 먼저 정하고,
화면과 셀 통합을 맡았습니다

인터페이스

콜백 교착을 확인한 뒤 서비스 12개를 팀 함수 계약 12개 · 메시지 2개로 바꿔 먼저 고정(최종 13개 · 3개) → 4명이 로봇 없이 병렬 개발

두산 API가 콜백 안에서 멈추는 문제를 피하는 구조(노드 2개 · 기능은 함수)로 결정

운영 화면

시작 · 일시정지 · 재개 · 중단 화면과 용기별 기록 DB → 무선으로 이은 두 PC 사이 화면 버튼 응답 7~96 ms · 기록 누락 0건

FastAPI · Next.js · SQLite 테이블 5개(용기별 기록 · 소모품 · KPI 등) · 누적 처리 지표(KPI)

무게 측정

측정 조건을 바꿔 가며 오차 원인(그리퍼 케이블 장력)을 규명 → 오차 90 g에서 +3 g로, 잔반 4/4 감지 · 빈 용기 오판 0/8

저울 없이 관절 토크에서 추정한 외력으로 측정 · 절차 확정

셀 통합

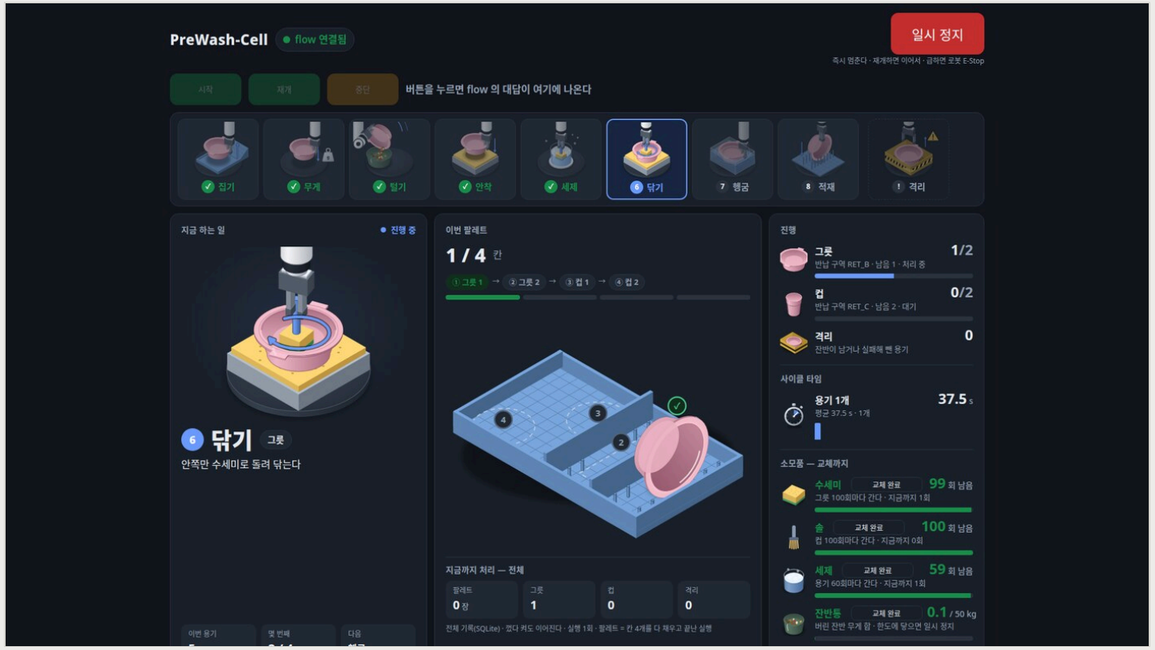
런치 패키지 · 공용 이동/정지 · 툴 집기/반납, 5일 차부터 통합 주도 → 정상 속도(1.0배속) 무정지 완주

지정 좌표 하강은 Z 직선 + 마지막 15 mm 완충으로 결정

PM

PR을 관리하고 결정 사항 71건을 기록, 기능 동결 뒤에는 "기본 흐름 유지 · 예외만 수정" 원칙으로 반영 → 수락 기준 7/7 충족

요구사항 · 인터페이스 · 설계 문서 · 6일 차 기능 동결



운영 화면 — 공정 단계 · 현재 작업 · 팔레트 상태 · 소모품 · 누적 처리 지표(KPI).

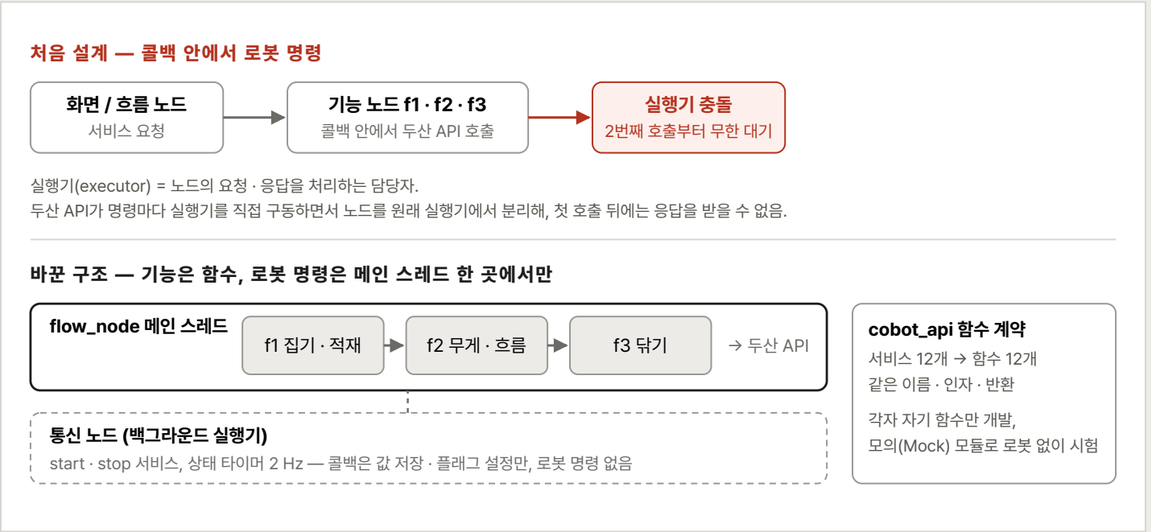
Problem.

콜백 안에서 로봇 명령이 두 번째부터 멈추는 문제

문제	**로봇 동작을 ROS 2 서비스 콜백 안에서 호출하자, 첫 명령은 되고 두 번째부터 응답 없이 무한 대기** 오류 메시지 없이 노드 · 로봇 · 로그는 정상으로 보여 원인을 찾기 어려움 · 한 번만 호출하는 단위 시험은 통과해 통합 때 드러남
원인	**두산 API는 모든 로봇 명령 안에서 응답이 올 때까지 ROS 2 실행기를 직접 구동함 (코드 241곳)** 콜백 안에서는 실행기가 이미 그 콜백을 처리 중이라 응답을 받을 수 없고, 첫 호출 뒤 노드가 실행기에서 빠져 두 번째부터 멈춤
재현	**모의(Mock) 드라이버로 같은 증상을 재현해 원인을 확정** 노드를 두산 API에 넘기고 다중 스레드 실행기로 실행해도 두 번째부터 멈춤 — 두 번 실행해 두 번 모두 두 번째 호출에서 정지
해결	**로봇을 움직이는 코드를 콜백 밖으로 — 기능은 함수로 만들고, 메인 스레드가 차례로 호출** 통신 노드는 값 저장 · 플래그(flag) 설정만 · 서비스 12개를 같은 이름의 함수 12개 계약으로 바꿔 4명이 로봇 없이 각자 개발
결과	**가상 로봇에서 다른 구조들과 같은 시험으로 비교해 가장 안정적인 구조임을 확인** 연속 6회 성공 · 이동 중 상태 발행 2.0 Hz · 이동 중 정지 수락 → 정상 속도(1.0배속) 무정지 완주에 사용

배운 점

- 로봇 명령은 **콜백 밖 한 곳에서만** 호출해야 합니다. 통신 노드에 값 저장 · 플래그 설정만 맡기면 교착이 생길 자리가 없습니다
- 한 번씩만 호출하는 단위 시험만으로는 **실행기 충돌** 같은 문제를 걸러 내지 못합니다. 병렬 개발에 들어가기 전에 **연속 호출 구조부터 검증**해야 합니다



처음 설계와 바꾼 구조의 차이.

Others.

그 밖의 프로젝트

학습

손 제스처로 켜고 끄는 물체 추적 로봇 — 상태 · 이동 제어 담당 · 2026.07 – 08 · 5인

손 제스처(보자기 = 시작, 주먹 = 정지)로 빨간 공 추적을 제어. 위치 · 면적 오차 비례 제어, 안전 정지 3종(정지 제스처 · 미검출 · 신호 끊김), 토픽별 통신 품질(QoS) 분리 · 제어 인터페이스 규약 확정 · turtlesim · Gazebo 검증 · github.com/hwang-injae/gesture-controlled-robot-tracking

실습

두산 M0609 기어 조립 — 티치 펜던트 프로그램(DRL)을 ROS 2 노드로 변환 · 2026.09 · 개인

실제 로봇에서 기어 4개 조립 · 홈 복귀까지 완주(약 44초). 달라진 대기 인자 · 조건 함수 반환값 · 힘 제어 중 흔들기 거부를 로그로 찾아 수정 · github.com/hwang-injae/rokey-mini-projects

실습

TurtleBot4 RC카 탐색 · 추적 — 객체 탐지 담당 · 2026.08 – 09 · 4인

고정 웹캠이 RC카를 발견하면 TurtleBot4가 출동해 탐색 · 추적. 웹캠용 RC카 · 더미 박스 인식 모델(YOLO11n) 학습과 출동 트리거 노드 담당, SLAM · 원격 조종 두 방법으로 지도 생성. 탐색 위치 이동까지 동작했으나, 무선 지연(ping 3~10초)으로 로봇 카메라 토픽이 PC에 늦게 도착해 실시간 탐지가 어려워 회전 탐색의 목표 고정은 다음 과제로 남김

프로젝트

IDC Patrol — 데이터센터 순찰 로봇 2대 · 데이터 수집 · 라벨링 기준 → PM · 2026.09 · 8인

TurtleBot4 2대가 구역을 나눠 순찰하며 열린 랙 문(YOLO)과 랙 번호(ArUco)를 판별해 관제 화면에 보고하는 6일 MVP. 라벨링 가이드 · 검수 규칙 작성과 데이터 수집 · 라벨링, PC 4대 구성도 · 파트별 구성도 3종, 요구사항 · 설계 문서 개정, 4일 차부터 데일리 브리핑으로 일정 관리를 맡음. 기능별 동작까지 확인했고 전체 통합 시연은 다음 단계로 남김 · github.com/hwang-injae/idc-patrol-robot

Thank you.

읽어 주셔서 감사합니다

네 번의 팀 프로젝트에서 센서 · 비전 판단 결과를 로봇 동작과 운영 화면으로 잇는 일을 맡으며,
부분을 합쳐 실제로 돌려 봐야 드러나는 문제를 찾아 고치는 기본기를 익혔습니다.

배운 것을 바로 써 보고 막히면 원인을 끝까지 파고들어,
현장에서 멈추지 않고 동작하는 로봇 SW를 만들겠습니다.

로봇을 움직이는 재미에 빠진 로봇 SW 개발자

황인재

EMAIL
il1282113@gmail.com

PHONE
010-5377-1314

GITHUB
github.com/hwang-injae 프로젝트별 README · 시연 영상

LINKEDIN
linkedin.com/in/hwang-injae